

Lab 3

Objective: Learn about Logisim-Evolution

In this lab, you will use Logisim-Evolution to build and test digital hardware. You will first construct a combinational adder, then complete a sign-extension circuit, and finally connect combinational logic with clocked registers to create a small accumulator.

1 Open the Lab Files

Complete the setup and course-update steps in the course repository README before starting. Open a terminal inside the course container and change to:

```
cd /workspace/labs/lab3
ls
```

2 Start Logisim-Evolution

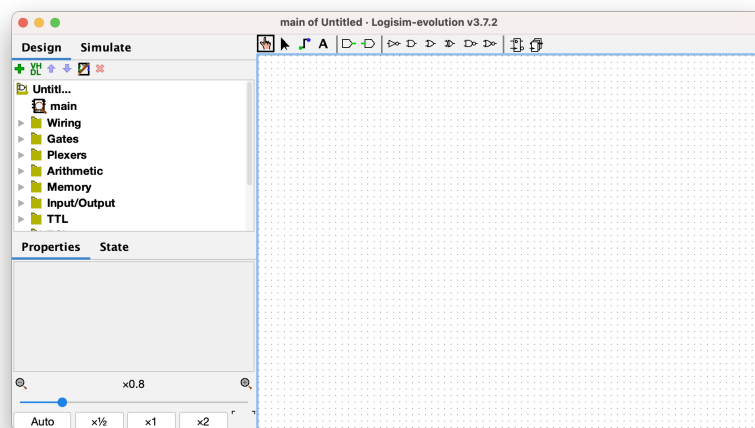
In the terminal inside the course container, run:

```
logisim &
```

Open the browser-desktop address printed by `./manage up`. Logisim-Evolution should appear in that desktop. The course's pinned Logisim-Evolution version and its Java runtime are already installed in the container.

3 Orienting Yourself

3.1 The Logisim-Evolution Interface The Logisim-Evolution interface should resemble the screen below. Minor visual differences are expected because the course container uses a newer pinned release.



3.2 Learn the Basics You can learn the basic operations of Logisim-Evolution from its built-in tutorial. Access the tutorial by selecting:

Help → Tutorial

For our purposes, make sure you go through the following sections:

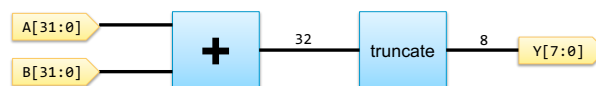
- Beginner's Tutorial
- Additional features → Creating Bundles
- Additional features → Splitters
- Additional features → Wire colors

3.3 Check Yourself

Make sure you know the answers to the following:

- How do you add an input pin to your circuit?
- What is the function of the Poke tool ()?
- Given a 32-bit signal, how do you split the signal and extract only the 8 least significant bits to form a new 8-bit signal?
- What is a “Tick”? How do you use that to test your circuit?

3.4 A Simple Adder Create and save a new circuit as `/workspace/labs/lab3/add32_8.circ`. The circuit takes two 32-bit values, $A[31:0]$ and $B[31:0]$, and adds them using a built-in 32-bit adder. Use a splitter to retain only the least-significant eight result bits as $Y[7:0]$. Connect a one-bit constant zero to the adder's carry-in input and leave carry-out unconnected.



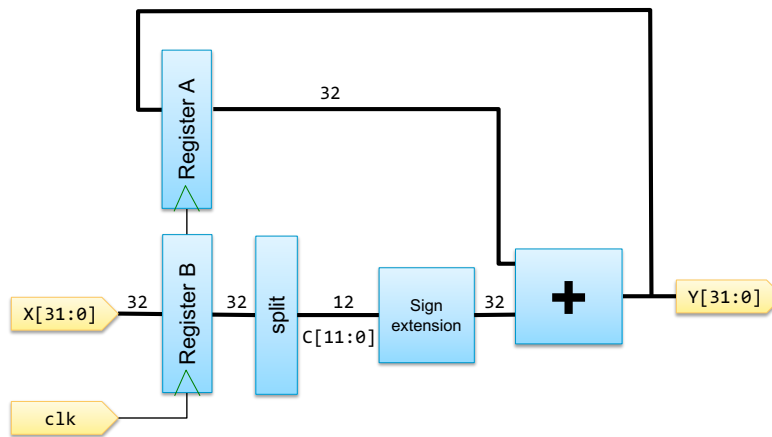
3.5 Checkoff 1

- Submit your adder circuit above.
- Show the circuit working for at least these input pairs: $1+2$, $0xff+1$, and $0x12345678+0x11111111$. Explain why only the least-significant byte appears at Y .

4 A Small Accumulator

Implement the following `small-accumulator` in Logisim-Evolution. The circuit takes a single 32-bit signal $X[31:0]$. Signal X is stored in a 32-bit register (Register B). The 12-bit field $X[19:8]$ is then extracted from Register B to form $C[11:0]$, which is expanded to 32 bits by a *sign-extension* block.

The sign-extended value of C is added to the current value of Register A to produce $Y[31:0]$. The result feeds back to Register A and is stored at the next active clock edge. Register A therefore holds the previous result, while the visible combinational Y includes the value currently stored in Register B.



4.1 Sign Extension In class, we discussed the operating principle of sign extension. Here, you will complete one such block in Logisim-Evolution:

- Open `/workspace/labs/lab3/smallacc.circ`.
- **Double-click** the `signext` circuit on the left, below `main`.

You should now see the sign extension circuit that is inside the main small accumulator circuit.

The `signext` circuit has one input `C[11:0]` and one 32-bit output `CX[31:0]`. The circuit is complete except for the values of the two **Constant** blocks.

Your task is to set the correct values for those blocks:

- Click each constant connected to the multiplexer.
- In the “Attribute Table” pane, change its **Value** from the default value to the required all-zero or all-one value.

The input splitter sends `C[10:0]` directly to the output and uses `C[11]` as the multiplexer selector. The multiplexer must therefore supply 21 bits: the original sign bit plus the 20 additional sign-extension bits. Test at least `C=0x001`, `0x7ff`, `0x800`, and `0xff`, paying particular attention to the signed boundary at `0x800`.

4.2 Complete Circuit Now, complete the rest of the circuit by:

- Open `smallacc.circ` if you have not done so.
- **Double-click** the `main` circuit. You should see a template circuit with all the necessary sub-blocks.

Connect `X` to Register B. Use a splitter to extract Register B bits `[19:8]` and connect the resulting 12-bit value to `signext`. Add the sign-extended result to Register A, connect the sum to `Y`, and feed it back to the input of Register A. Both registers must use the same clock. Connect a one-bit constant zero to the adder’s carry-in input.

4.3 Checkoff 2

Implement and test your small accumulator circuit. Submit:

- Your completed circuit file (`smallacc.circ`).
- Screenshots showing how values accumulate on each cycle. First, hold `X=0x00000100` for three full cycles. Then reset and apply `0x00000500`, `0x000ffe00`, and `0x00000100`, one full cycle each. Explain the signed 12-bit value contributed by each input.

4.4 Submission Submit your answers to Checkoffs 1 and 2 above.

4.5 Going Beyond Compare this circuit with the single-cycle datapath: Register A plays a role similar to the register file, Register B to the PC, the adder to the ALU, and the splitter to instruction decode. Explain the similarities and the limits of these analogies.